Canonical witnesses for modular convex subsequences with a small index deficit

Stamatis-Christos Saridakis

24 October 2026

Research draft. This technical note states a specialized upper bound for a corrected research implementation. Literature priority and suitability for a publication venue remain unresolved.

Abstract

Consider a maximum-cardinality subsequence of an integer array whose original indices sum to zero modulo a prescribed integer and whose consecutive selected values have strictly increasing differences. We study the regime where the modulus is close to the sum of all indices. Writing $h = n(n+1)/2 - m < n$ and $H = h + 1$, we prove that the globally lexicographically least entire optimal index tuple can be computed in $O(n + H^{5/2} \log^2(h+2))$ wide-word operations using $O(H^2)$ auxiliary words, excluding the input and one returned output. The algorithm combines established sorted-difference dynamic programming and canonical self-reduction with a deficit-specific reduction. The omitted indices sum to h, giving a core of at most $h + 2$ indices, $O(H \log(h+2))$ eligible transitions, and at most $O(\sqrt{H})$ omissions. A forced-prefix scalar oracle and monotone threshold queries reconstruct the complete canonical witness without storing witness histories. An accompanying computational audit checks 80,283 complete-witness instances and identifies a linear auxiliary allocation in the original implementation's final validation; an indexed validation and deterministic output cursor repair it. The contribution is a specialized composition with explicit complexity accounting, without a claim of a new general dynamic-programming primitive or established literature priority.

1 Introduction

An optimal score and a canonical optimal witness impose different obligations on an exact algorithm. For a convex subsequence, a scalar score can be propagated in sorted order of value differences. Retaining one locally preferred predecessor does not by itself show that the resulting entire index tuple is globally lexicographically least. A second difficulty appears in space analysis: even when the optimization state is small, copying a large returned witness during validation can invalidate an auxiliary-space bound.

This note addresses both obligations for a modular index constraint in a restricted parameter regime. The input is an integer array $A[1..n]$. Selected values must be strictly convex in their selected order, and the sum of the selected original indices must be divisible by m. The objective first maximizes the number of selected indices and then minimizes their complete increasing tuple lexicographically.

If m is sufficiently large, the modular condition becomes an exact omitted-weight condition. When its deficit h is smaller than n, all choices occur near the start of the array.

The main result is a complete algorithm and proof with quadratic auxiliary space in $h+1$. Its central reconstruction step selects omissions rather than testing every input position: an optimal omitted set has only $O(\sqrt{h+1})$ elements, and each next omission is found by binary search on a threshold feasibility predicate. The proof accounts for every repeated query, the compulsory suffix, event sorting, and output construction.

The mathematical mechanisms have clear antecedents. Accordingly, the intended contribution is a specialized bound and a reproducible audit of its implementation. The manuscript does not claim a general advance in canonical traceback, a lower-bound-optimal algorithm, or a demonstrated practical speedup. The corrected artifact and the original implementation are preserved separately in the MOCOSO research repository (Saridakis 2026).

2 Problem and computational model

Let $n \geq 2$, let $A[1..n]$ contain arbitrary signed integers, and let $m \geq 2$ be an integer. Define

$$U = \frac{n(n+1)}{2}, \qquad h = U - m, \qquad H = h + 1.$$

Assume $U/2 < m \leq U$ and $0 \leq h < n$. A feasible witness is a nonempty tuple $S = (s_1, \ldots, s_k)$ with strictly increasing entries in $[n] = \{1, \ldots, n\}$ satisfying

$$\sum_{r=1}^{k} s_r \equiv 0 \pmod{m},$$

and, for each $1 \leq r \leq k-2$,

$$A[s_{r+2}] - A[s_{r+1}] > A[s_{r+1}] - A[s_r].$$

Tuples of length one or two satisfy the convexity condition vacuously. This is convexity of selected values in selected order. It is not a condition on divided differences using the original index gaps. Among all feasible witnesses, the requested output has maximum length and, among witnesses of that length, is lexicographically least. An empty tuple denotes infeasibility and is not a feasible candidate of score zero.

We count operations on wide words large enough to contain input values, differences, triangular sums, addresses, and polynomially bounded counters. If signed input values require at most B bits, $b = \Theta(B + \log(n+2))$ bits per word suffice, with an adequate constant factor. Arithmetic and comparisons on such words have unit cost in the primary theorem. This is not a fixed 64-bit RAM assumption. The auxiliary-space measure excludes a read-only random-access input and one final output array; all intermediate arrays, event records, sorting storage, and temporary output copies are charged.

Theorem 1. Under these assumptions, the globally lexicographically least entire maximum-cardinality feasible witness, or an infeasibility report, is computable deterministically in $O(n + H^{5/2}\log^2(h+2))$ wide-word operations and $O(H^2)$ auxiliary wide words.

The theorem concerns the algorithm below and the corrected research derivative described in Section 7. The original pinned implementation has a separate auxiliary-space defect. With elementary bit arithmetic, a conservative translation of the theorem gives time $O((n + H^{5/2} \log^2(h + 2))b^2)$ and auxiliary storage $O(H^2 b)$ bits. The excluded input and output require $O(nB)$ and $O(n \log(n + 2))$ bits, respectively. These bounds do not predict Python process RSS or allocator behavior.

3 Established ingredients and scope of contribution

The unweighted value-convex subsequence problem appears in Erickson's algorithms textbook (Erickson 2019, 126, exercise (f)). Kluev's 2013 public technical answer gives the closely related sorted-difference sweep, including decreasing destination indices for equal differences, scalar endpoint scores, and witness reconstruction suggestions (Kluev 2013, steps 1–6). That answer is a primary public disclosure, not a peer-reviewed paper. The sweep and its equal-difference ordering are therefore established ingredients, not claimed contributions of this note.

Exact resource coordinates in dynamic programming are also standard. Turner describes equality-constrained resource shortest paths on acyclic digraphs and gives a pseudopolynomial algorithm (Turner 2011, equation (23), Theorem 2). Her result supplies an appropriate resource-DP template, but not the compressed event invariant or the deficit-dependent transition count needed here. Simply expanding a last-two-indices graph would not establish our stated bound.

Canonical witness search by constrained extension queries belongs to established self-reduction methods; Große, Rothe, and Wechsung explicitly discuss lexicographically first solutions in that setting (Große, Rothe, and Wechsung 2006, sec. 3). Our use of threshold queries needs a proof that the particular constrained optimization oracle remains efficient. Unconstrained decision alone is insufficient for that conclusion. Low-space exact resource algorithms also occur in Subset Sum (Jin, Vyas, and Williams 2020); their problem does not include selected-order convexity or this entire-index-tuple objective, so no black-box implication is asserted here.

The surviving contribution is the joint specialized analysis: an exact omission deficit, a two-anchor core, a harmonic eligible-event bound, a constrained scalar sweep, and sparse omission reconstruction with all queries charged. These statements can be derived by combining familiar mechanisms with elementary problem structure. We did not identify the same complete theorem in the sources inspected on 24 October 2026. That bounded observation does not establish firstness, and an earlier equivalent result would substantially weaken the paper's originality claim.

4 Deficit structure and eligible transitions

Lemma 2 (exact complement). For any feasible witness, its omitted set D, consisting of the indices outside S, satisfies $\sum_{d \in D} d = h$. Conversely, any complement with that omitted sum satisfies the modular index condition. Every omission is at most h, and the omission count t satisfies $t(t + 1)/2 \le h$.

Proof. A nonempty selected sum is positive and at most $U < 2m$. Its only possible positive multiple of m is m. Taking complements gives omitted sum $U - m = h$, and the converse follows immediately. An omitted index greater than h would already exceed the nonnegative total. Distinct positive omitted indices have sum at least $1 + \cdots + t = t(t + 1)/2$. Thus $t = O(\sqrt{H})$. In particular, every index from $h + 1$ through n is compulsory. □

Set $p = \min(n, h+2)$ and let $W(q) = q(q+1)/2$ for $q \geq 0$.

Lemma 3 (two-anchor core). Test the original consecutive triples ending at indices $k = h+3, \ldots, n$. If any fails strict convexity, the instance is infeasible. Otherwise, feasible core witnesses on $[p]$ that include its compulsory indices correspond exactly to full feasible witnesses by appending $p+1, \ldots, n$. Scores increase by $n-p$.

Proof. Each tested triple consists entirely of compulsory indices and hence occurs consecutively in every witness. When $p < n$, the core contains the two compulsory indices $h+1, h+2$. Consequently, the first triple crossing into the appended suffix ends at $h+3$ and is among those tested. Every later suffix triple is tested as well. Any triple involving a mutable index and ending at either compulsory anchor remains in the core. If $p = n$, there is no suffix to append. Restriction and extension therefore preserve feasibility in both directions. □

The suffix check costs $O(n)$ once. Including only $h+1$ as an anchor would leave a triple involving the last mutable selected index unchecked; the second anchor is essential to this reduction.

A core event is a possible pair of consecutive selected indices (i, j) with $1 \leq i < j \leq p$. Retain the pair only if its open gap contains no compulsory index and its omitted weight

$$g(i, j) = W(j-1) - W(i)$$

is at most h. Adjacent indices have gap weight zero. Every consecutive selected pair of a feasible core witness is eligible. For each retained event, store its difference $d(i, j) = A[j] - A[i]$ and its gap weight.

Lemma 4 (harmonic event count). The number M of eligible events is $O(H \log(h+2))$. They can be enumerated in $O(M+p)$ time and sorted in $O(M \log(M+1))$ time.

Proof. There are at most $p-1$ adjacent events. For a nonempty gap containing $u = j - i - 1$ indices, its weight is

$$g(i, j) = ui + \frac{u(u+1)}{2}.$$

Eligibility implies $u \leq \text{floor}(\sqrt{2h})$ and $i \leq \text{floor}(h/u)$. Hence

$$M \leq p - 1 + \sum_{u=1}^{\text{floor}(\sqrt{2h})} \text{floor}(h/u) = O(H \log(h+2)).$$

This is an upper bound, not an assertion that every counted pair is eligible. For each fixed i, increasing j increases the gap weight and cannot remove a compulsory-index violation. Thus a direct scan stops at its first failure, charging eligible events and at most one failed candidate per i. Comparison sorting gives the claimed preprocessing cost. When $h = 0$, the sum is empty and the bounds remain valid. □

5 A forced-prefix scalar optimization oracle

Fix a set $Q \subseteq [h]$ of forced omissions and a threshold $x \in [1, h+1]$. The oracle $F(Q, x)$ returns the maximum full-witness length subject to the original constraints, omission of every member of Q, and selection of every index less than x outside Q. It returns zero if no such witness exists. The suffix check from Lemma 3 is performed before any oracle calls.

Let a Boolean array mark forbidden endpoints in Q, and define a prefix count

$$R(v) = |\{r \le v : r < x,\ r \notin Q\}|, \qquad R(0) = 0.$$

An event (i, j) is allowed for this query precisely when neither endpoint is forbidden and $R(j-1) = R(i)$, so its gap contains no required index. These checks take constant time per event. A singleton start at i is allowed when $i \le \min(p, h+1)$, $i \notin Q$, $W(i-1) \le h$, and $R(i-1) = 0$.

Create a matrix $V[i, r]$ for $1 \le i \le p$ and $0 \le r \le h$, initially unreachable. For each allowed singleton start, set $V[i, W(i-1)] = 1$. Sort all eligible events once, outside the oracle, by increasing difference and then decreasing destination. Remaining ties may be broken arbitrarily. For each allowed event (i, j), and for each $0 \le r \le h - g(i, j)$ with reachable $V[i, r]$, perform

$$V[j, r + g(i, j)] \leftarrow \max\{V[j, r + g(i, j)], V[i, r] + 1\}.$$

The result is $V[p, h] + n - p$ if $V[p, h]$ is reachable, and zero otherwise. The matrix stores only lengths. No predecessor tuple or persistent witness tree is retained.

Lemma 5 (event-relative dominance). Immediately before an event (i, j) of difference d is processed, the source values $V[i, r]$ represent optimal lengths among admissible paths ending at i, of omitted weight r through i, whose last difference is less than d, together with admissible singleton starts. The update therefore enforces strict convexity, and the final oracle value is correct.

Proof. All events with difference less than d have been processed. Any event of difference exactly d entering i has destination $i < j$, and decreasing destination order places it after the current event. Thus no equal-difference incoming event can yet have contributed to $V[i, r]$. Events with larger difference also have not been processed. Singleton starts have no last difference and may precede any first edge.

Inductively, every reachable state arises from a strictly convex selected path satisfying the endpoint and required-gap checks. Conversely, the previous edge of any admissible path that can use (i, j) has smaller difference and has already been processed, so that path's predecessor score is represented or dominated. At this event, a larger score with the same endpoint and exact resource can use every continuation available to the smaller score: the source paths all satisfy the current difference threshold. Retaining only the maximum score therefore loses no optimal extension. This reasoning is relative to the event order; it does not claim that arbitrary endpoint-equal paths with different last differences are interchangeable at all times.

Singleton omission weights and the disjoint open gaps count exactly the indices omitted through the current endpoint. Forbidden endpoints ensure that members of Q are absent, and required-prefix checks ensure that every required index is present. Eligibility prevents compulsory indices from being skipped, including by an initial start. Since p is compulsory, an admissible core path

must end at p and have total omitted weight h. Lemma 3 then supplies exactly the appended suffix length. This proves both soundness and completeness. □

Lemma 6 (oracle resources). A query costs $O(H^2 \log(h+2))$ operations and $O(H^2)$ auxiliary words.

Proof. The matrix has $O(pH) = O(H^2)$ entries. Its initialization, flags, and prefix counts take $O(H^2)$ time. Each of M events scans at most H resource entries, for $O(MH)$ time. The event list occupies $O(M)$ words and is reused. The matrix is released between calls. Lemma 4 gives the bounds. There is no suffix or output scan inside a query. □

6 Canonical reconstruction and total complexity

Compute $L = F(\emptyset, 1)$. If $L = 0$, report infeasibility. Otherwise all optimal witnesses omit exactly $t = n - L$ indices. Maintain an increasing omission prefix $Q = (q_1, \ldots, q_r)$ that is the beginning of the omitted tuple of at least one optimum. Initially Q is empty and its previous index b is zero. Until $r = t$, binary-search thresholds $x \in [b+1, h+1]$ for the greatest value satisfying

$$F(Q, x) = L.$$

Append that value to Q and update b. Return the increasing complement of the completed Q.

Lemma 7 (threshold monotonicity and extension). With the stated prefix invariant and $r < t$, the predicate $F(Q, x) = L$ is true exactly when some optimal omitted tuple extending Q has its next element at least x. It is true at $b+1$, false at $h+1$, and monotone from true to false. Its greatest true threshold is the greatest possible next omission among those optima.

Proof. Every index below x outside Q must be selected. Since $x > b$ and all members of Q are forbidden, this fixes the omission prefix through $x-1$ to exactly Q. There remains at least one omission because $r < t$. It must lie at or beyond x. Conversely, an optimum with that omission prefix and next omission at least x satisfies all query constraints. Constrained scores cannot exceed the unconstrained optimum L, so equality is exactly the claimed existence test.

Increasing x adds required selections and cannot create an optimum. The prefix invariant gives truth at $b+1$. At $h+1$, every mutable index outside Q would be required, leaving no location for the remaining omission. Thus the upper endpoint is false. At a greatest true threshold x^*, a witnessing optimum cannot have its next omission greater than x^*, because it would then satisfy threshold x^*+1 as well. Its next omission is therefore exactly x^*. Appending it preserves the prefix invariant and makes the next omission as large as possible. □

This predicate concerns a lower bound on the next omission, not equality to a proposed omission. Equality feasibility need not be monotone. For example, with $A = (1, 3, 9, 27, 81, 243, 729)$, $n = 7$, and $m = 22$, the maximum length is six and the only optimal omission is index six. The equality predicate is false at one through five, true at six, and false at seven. The threshold predicate is true at one through six and false at seven.

Lemma 8 (entire-witness order). Lexicographically maximizing the increasing omitted tuple among optimal witnesses lexicographically minimizes the entire selected tuple.

Proof. Compare distinct omitted sets of equal cardinality, and let z be the smallest index in their symmetric difference. Their increasing omitted tuples first differ in favor of the set that contains z:

that tuple is lexicographically smaller. The selected complements first differ in the reverse direction, since exactly the other complement contains z. Thus complementation reverses the order. Lemma 7 successively maximizes every omission coordinate among surviving optima, so the completed omitted tuple is globally greatest. Its complement is globally least. □

Proof of Theorem 1. Correctness follows from Lemmas 2–8. For an explicit query bound, count the initial optimization, one lower-endpoint assertion query per omission, and at most $\mathrm{ceil}(\log_2 H)$ binary-search queries per omission. This yields

$$q \le 1 + t(1 + \mathrm{ceil}(\log_2 H)).$$

For $h = 0$, $t = 0$, so only the initial query is needed. In all cases, Lemma 2 gives $q = O(1 + \sqrt{H}\log(h+2))$. Multiplying by Lemma 6 gives the claimed parameter-dependent time. Sorting the single event list costs $O(H \log^2(h+2))$, which is absorbed. The initial input and suffix checks and final output construction cost $O(n)$ in total; they are not repeated by the oracle.

At any instant there is one $O(H^2)$ matrix, an $O(H\log(h+2))$ event list with sorting workspace of the same order, $O(H)$ flags and prefix counters, and $O(\sqrt{H})$ chosen omissions. These occupy $O(H^2)$ auxiliary words. Allocate the known-length output once and fill it by scanning $[n]$ against a monotone cursor in the sorted omission list. Validate it by indexed reads with a constant number of temporary words. No additional output-sized tuple or list is necessary. This establishes the stated auxiliary-space bound. □

7 Implementation defect and correction

The original G9 source is pinned to repository commit `96ac7e2` (full identifiers are in the evidence manifest). Its final convexity assertion iterates over `zip(selected, selected[1:], selected[2:])`. Python evaluates both slices before the iteration, creating two additional tuples of length proportional to the output. These tuples are auxiliary storage even though the original returned tuple is excluded.

The counterfamily is $h = 0$, $m = U$, and $A[i] = i^2$. Every index must be selected and the complete array is strictly convex. Therefore $H = 1$ while the two slices use $\Theta(n)$ auxiliary reference slots. The original code's asserted $O(H^2)$ auxiliary-space bound is false on this family. Passing functional tests cannot detect that asymptotic defect without a space-specific check.

The corrected derivative replaces the sliced assertion with indexed comparisons and constructs the output through an omission cursor. The cursor also removes reliance on average-case hash membership for a deterministic output-time claim. Its iterator exposes the known remaining output length to CPython; interpreter allocation details remain outside the abstract storage theorem. The scalar DP, event order, and reconstruction loop are unchanged. The derivative is an attributed correction, not a clean-room reimplementation of G9.

Both versions and all prior research implementations are retained in snapshot `43353e2` (Saridakis 2026). The corrected file is `research/v03/g9_independent_audit/solver_corrected.py`. Its complete SHA-256 and the audit harness hash appear in the companion evidence manifest, avoiding ambiguity about which implementation supports the theorem.

8 Computational audit

The computational evidence supplements the proof. The preserved audit uses seed 2026102415 and CPython 3.12.14 on Linux x86-64. Three separately written reference approaches were used: direct selected-subset enumeration, full-array dynamic programming indexed by the last two selected indices and exact selected sum, and distinct-part omission enumeration for planted instances. The reference methods do not import upstream reference solvers. Independence here means separately written methods within the same assisted audit session, not blinded external replication or formal verification.

The full-witness corpus contains 80,283 instances. The following categories are disjoint; the additional pair-DP checks are a subset of the random category.

Full-witness category	Instances	Scope
Exhaustive selected-subset comparisons	73,809	Three values, lengths 2 through 8, valid deficits
Seeded random comparisons	6,000	Lengths 3 through 12, six input families
Planted omission instances	450	Deficits 0 through 34
Maximum-omission stress cases	6	Up to 16 omissions
Constructed deep lexicographic ties	18	Shared omission prefixes of lengths 0 through 5
Total	80,283	Complete returned tuples compared

The exhaustive alphabet is $\{-3, 0, 5\}$. Random families include small signed values, perturbed quadratics, random walks, and very large integers, including 120-digit magnitudes. Of the 6,000 random instances, 1,800 also receive the independent last-two-indices DP check. Across the full corpus, 75,621 instances are infeasible and 4,662 feasible. This imbalance limits any interpretation as a representative workload; the suite is designed for correctness falsification, not performance estimation.

The constrained oracle is tested separately on 13,767 forced-omission/threshold queries against direct enumeration; 1,565 of those also receive the independent pair-DP check. Event enumeration and its harmonic bound are checked for all deficits from zero through 200. These counts are additional checks, not additional full-witness instances. No mismatch is recorded for the original or corrected exact returned witnesses in the preserved run.

Mutation tests deliberately reverse the equal-difference destination order, remove forbidden-endpoint or required-prefix checks, and shorten the core to one anchor. Concrete counterexamples in the audit artifact show that these changes break the relevant invariants. For instance, with all four values zero and $m = 8$, ascending equal-difference order can return $(1, 3, 4)$ even though no feasible witness exists. The mutant's assertions are disabled only to observe that invalid result; assertions remain enabled for the normal audit.

An additional search over 10,000 random inputs found no hole in the feasible interval of first omissions under the investigated hypothesis. That failed falsification attempt establishes no interval

theorem and is not used in the proof. The proof uses the nested threshold constraints directly. All explicit mutation examples and the failed-search scope are preserved in the audit report.

The validation-phase allocation probe isolates the discovered slice defect. After the returned tuple is constructed, it resets the tracing peak and measures extra allocations during validation and return. It does not measure total solver memory or RSS.

Input length	Original extra bytes	Corrected extra bytes
1,000	17,160	1,022
10,000	160,992	832
100,000	1,601,000	840

These observations are consistent with the explicit allocation argument in Section 7. Their precise byte counts depend on the runtime and do not establish a machine-independent empirical complexity law. The auxiliary bound for the corrected algorithm follows from object accounting in Theorem 1.

9 Discussion and limitations

The omission representation is useful because it changes the number of reconstruction decisions. A direct feasibility decision for each mutable index makes $O(H)$ oracle calls. This reconstruction uses $O(\sqrt{H}\log(h+2))$ calls while retaining scalar states. Its cost includes an additional logarithm relative to the repository's stated G8 sparse-witness time bound. In that comparison, the G8 analysis states time $O(n + H^{5/2}\log(h+2))$ and auxiliary space $O(H^2\log(h+2))$, whereas the present theorem removes the logarithm from auxiliary space at the cost of another logarithm in time (Saridakis 2026, G8 theorem laboratory). This is a comparison with a repository-stated theoretical bound, not independent certification of every allocation in the G8 implementation, and not an external state-of-the-art ranking.

Five conclusions must be kept separate. Mathematical correctness is supported by the explicit derivation. Implementation correctness includes a verified space defect in the original and a corresponding correction. The asymptotic contribution is a specialized space-time tradeoff. Literature priority is unresolved. Scientific significance is bounded by the narrow regime, established ingredients, and absence of demonstrated applications or lower bounds. None of these conclusions follows automatically from any of the others.

Several limitations remain. The algorithm assumes $h < n$ and does not solve all modulus regimes within these bounds. Arithmetic on arbitrary-size values is charged through a wide-word model. The experiments are finite, predominantly small, and not an industrial benchmark. The independent reference programs and the proof were prepared in an AI-assisted workflow within the same session; external expert replication remains valuable. The literature search is source-specific but not exhaustive. The correctness theorem can survive a priority discovery even if the paper's novelty case does not.

Useful subsequent work would identify a substantive application or a broader parameter regime, prove stronger tradeoffs, or establish a meaningful lower bound. Such possibilities are research questions, not results of this note. A venue decision should depend on the surviving specialized theorem and the reproducible correction, not on the availability of polished manuscript files.

10 Conclusion

In the high-modulus small-deficit regime, exact modular feasibility confines all omissions to a short prefix and bounds their number by a square root of the deficit. A constrained sorted-difference scalar DP then supports monotone searches for the lexicographically greatest omitted tuple, whose complement is the globally least complete optimal witness. Charging all searches and output operations gives quadratic auxiliary wide-word space and $O(n + (h+1)^{5/2}\log^2(h+2))$ time for the corrected algorithm. The result is a specialized composition of established methods with a documented implementation correction; its literature priority and publication significance remain open to further assessment.

Artifact availability and disclosure

The reproducible audit snapshot is linked in the references (Saridakis 2026). From that commit, run `python research/v03/g9_independent_audit/audit.py` and `python -m unittest tests.test_v03_g9_quadratic_lex -v` with assertions enabled. The first command regenerates `results.json`; use a disposable checkout to preserve the committed evidence. The companion manuscript folder includes the claim-evidence map, source registry, review, contribution stress test, evidence hashes, and build instructions. Original G3–G9 research files are preserved.

This manuscript was drafted with AI assistance, including source inspection, mathematical reconstruction, code execution, literature searches, and adversarial review. That review is an internal assisted critique, not external peer review. The named author's review and responsibility for any eventual submission are required; no affiliation, funding award, external endorsement, or venue acceptance is asserted.

References

Erickson, Jeff. 2019. *Algorithms*. 1st ed. Self-published. https://jeffe.cs.illinois.edu/teaching/algorithms/.

Große, André, Jörg Rothe, and Gerd Wechsung. 2006. "A Note on the Complexity of Computing the Smallest Four-Coloring of Planar Graphs." https://arxiv.org/abs/cs/0106045v2.

Jin, Ce, Nikhil Vyas, and Ryan Williams. 2020. "Fast Low-Space Algorithms for Subset Sum." https://arxiv.org/abs/2011.03819.

Kluev, Evgeny. 2013. "Answer to Longest convex subsequence in an array." Stack Overflow. https://stackoverflow.com/questions/20664920/longest-convex-subsequence-in-an-array.

Saridakis, Stamatis-Christos. 2026. "MOCOSO Research Artifacts and Independent G9 Audit." https://github.com/SaridakisStamatisChristos/MOCOSO/tree/43353e2b9a659d736b627788ec0c46554a090c75/research/v03.

Turner, Lara. 2011. "Variants of the Shortest Path Problem." *Algorithmic Operations Research* 6: 91–104. https://journals.lib.unb.ca/index.php/AOR/article/download/18312/20525/24773.